Problem Set Errors 2: Worked Solutions

Packet error probability · Hamming distance · CRC · Bit stuffing · Hamming code (Data Link Errors lecture, slides 7–8 and 28–45)

Q1. Maximum packet size for 99.99% success

> 5 intermediate nodes, every link has a bit error probability of 1 in ten million. The aim is to receive 99.99% of packets. What is the maximum packet size in bytes?

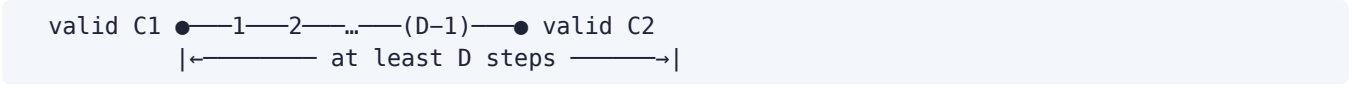
1. **Hops** = 5 + 1 = **6**. The packet is exposed on all 6 links, so the total number of bit-chances is 6 × 8L = **48L** (L in bytes).
2. **Condition:** P(good) = $(1 - 10^{-7})^{48L} \geq 0.9999$, which is the same as P(error) ≤ 10^{-4}.
3. **Solve:** 48L ≤ ln(0.9999) ÷ ln(1 − 10^{-7}) = 1,000 bits. (The approximation gives the same: 10^{-4} ÷ 10^{-7} = 1,000, because the target is tiny.)
4. L ≤ 1,000 ÷ 48 = 20.8, rounded **down** to 20 bytes.
5. **Check:** 20 B gives 960 bits × 10^{-7} = 0.96 × 10^{-4} ✓. 21 B gives 1,008 bits, which is 1.008 × 10^{-4} ✗.

20 bytes

Q2. Why minimum distance D gives "detect ≤ D − 1" and "correct < D/2"

> Give a plausible explanation (not a proof), based on the definition of Hamming distance and the way the scheme works.

Background: the Hamming distance between two words is the number of bit positions where they differ. Each bit error moves the received word **one step** away from what was sent. The minimum distance D means every pair of valid codewords is **at least D steps apart**. The receiver picks the **closest valid codeword** (slide 46).



Detecting ≤ D − 1 errors. Starting from the sent codeword C1, each error moves one step. With D − 1 or fewer errors, you can't reach another valid codeword, because that needs at least D steps. So the received word is **not** on the list of valid codewords, and the receiver knows an error happened. D errors could land exactly on another valid codeword, and that would go undetected.

Correcting < D/2 errors. If fewer than D/2 errors happen, the received word W is less than halfway from C1 to any other codeword. W is within D/2 of C1, and any other codeword C2 is at least D from C1, so C2 must be more than D/2 from W. That makes C1 the **closest** codeword, and nearest-codeword decoding picks the right one. With D/2 or more errors, W can be as close to (or closer to) a wrong codeword, and correction can fail.

> **Examples from the slides:** single parity has D = 2, so it detects 1 error and corrects none. 2-D parity has D = 4, so it detects 3 and corrects 1. The 4-codeword example has D = 5, so it detects 4 and corrects 2. With 3 errors it fails (slide 48).

Errors move the word one step at a time. Fewer than D steps can't reach another valid codeword, so the error is detected. Fewer than D/2 steps leave it closest to the original, so it's corrected.

CRC method (slides 39–41): G has n + 1 bits, so the CRC has n bits. **Sender:** append n zeros to M, divide by G using XOR (no borrows), and the remainder is the CRC. Send M followed by the CRC. **Receiver:** divide

everything it received by G. A remainder of 0 means no error was detected; anything else means an error was detected. XOR rule: 0⊕0 = 0, 1⊕1 = 0, 1⊕0 = 0⊕1 = 1.

Q3. 4-bit CRC: G = 10101, M = 1010011001001

1. G has 5 bits, so the CRC is 4 bits. Append 4 zeros: `1010011001001 0000`.
2. Divide. At each step, line G up under the first remaining 1 and XOR:

Step	Bits (window being divided in blue)	XOR with G
1	**10100**110010010000	10100 ⊕ 10101 = 00001
2	0000**11100**10010000	11100 ⊕ 10101 = 01001
3	00000**10011**0010000	10011 ⊕ 10101 = 00110
4	0000000**11000**10000	11000 ⊕ 10101 = 01101
5	00000000**11011**0000	11011 ⊕ 10101 = 01110
6	000000000**11100**000	11100 ⊕ 10101 = 01001
7	0000000000**10010**00	10010 ⊕ 10101 = 00111
8	00000000000**11100**	11100 ⊕ 10101 = 01001
end	000000000000**1001**	remainder = **1001**

3. Transmitted: `1010011001001 1001`. Check: dividing this by G gives remainder 0000 ✓.

CRC = 1001

Q4. 5-bit CRC: G = 101001, M = 101001100100101

1. G has 6 bits, so the CRC is 5 bits. Append 5 zeros: `101001100100101 00000`.
2. Divide:

Step	Bits (window being divided in blue)	XOR with G
1	**101001**10010010100000	101001 ⊕ 101001 = 000000
2	000000**100100**10100000	100100 ⊕ 101001 = 001101
3	00000000**110110**100000	110110 ⊕ 101001 = 011111
4	000000000**111111**00000	111111 ⊕ 101001 = 010110
5	0000000000**101100**0000	101100 ⊕ 101001 = 000101
6	00000000000000**101000**0	101000 ⊕ 101001 = 000001
end	000000000000000**00010**	remainder = **00010**

3. Transmitted: `101001100100101 00010`. Check: remainder 00000 ✓.

CRC = 00010

Q5. Receiver check: received 101011100101010011, G = 10011

1. The receiver does **not** add zeros. It divides exactly what it received by G.

Step	Bits (window being divided in blue)	XOR with G
1	**10101**1100101010011	10101 ⊕ 10011 = 00110
2	00**11011**00101010011	11011 ⊕ 10011 = 01000
3	000**10000**0101010011	10000 ⊕ 10011 = 00011
4	000000**11010**1010011	11010 ⊕ 10011 = 01001
5	0000000**10011**010011	10011 ⊕ 10011 = 00000
6	00000000000000**10011**	10011 ⊕ 10011 = 00000
end	000000000000000**0000**	remainder = **0000**

Remainder = 0000, so no error is detected

Q6. Receiver check: received 1011011101010111111, G = 11001

Step	Bits (window being divided in blue)	XOR with G
1	**10110**1110101011111	10110 ⊕ 11001 = 01111
2	0**11111**110101011111	11111 ⊕ 11001 = 00110
3	000**11011**0101011111	11011 ⊕ 11001 = 00010
4	000000**10010**1011111	10010 ⊕ 11001 = 01011
5	0000000**10111**011111	10111 ⊕ 11001 = 01110
6	00000000**11100**11111	11100 ⊕ 11001 = 00101
7	0000000000**10111**111	10111 ⊕ 11001 = 01110
8	000000000000**11101**11	11101 ⊕ 11001 = 00100
9	0000000000000**10011**	10011 ⊕ 11001 = 01010
end	00000000000000**1010**	remainder = **1010**

Remainder = 1010, which is not zero, so an error is detected

Q7. Bit stuffing and flags

Data before stuffing: 1110011111111000010110000001000110110111110100011110111111111111000

1. **Rule (slides 44–45):** scan the data. After every **five 1s in a row**, insert a **0**, even if the next data bit is already 0. Then restart the count. This stops the flag pattern 01111110 from appearing inside the data.
2. Scan from left to right. Each stuffed 0 is shown in **red** below. Four 0s are stuffed in total.

```
111001111101110000101100000010001101101111101000111101111111111000
```

3. Add the flag 01111110 at the start and at the end. The flags themselves are **not** stuffed.

```
01111110 111001111101110000101100000010001101101111101000111101111101111101000 01111110
```

The long run of ten 1s near the end becomes 11111**0**11111**0**: the count restarts after each stuffed bit. The receiver removes any 0 that follows five 1s, which gives back the original data.

Frame = 01111110 + stuffed data (65 data bits + 4 stuffed 0s = 69 bits) + 01111110

Hamming code layout (slides 49–53): 11 positions, numbered 11 down to 1. Redundancy bits sit at the powers of 2: **r1, r2, r4, r8**. The 7 data bits fill positions 11, 10, 9, 7, 6, 5, 3 from left to right. **r1** checks positions 3, 5, 7, 9, 11 · **r2** checks 3, 6, 7, 10, 11 · **r4** checks 5, 6, 7 · **r8** checks 9, 10, 11. Each r is set so that its group has an **even** number of 1s. At the receiver, the new parity bits (r8 r4 r2 r1) read as a binary number give the **position of the error**, and 0 means no error.

Q8. Hamming codeword for data 1011101

1. Place the data bits (r = still to compute):

Position	11	10	9	8	7	6	5	4	3	2	1
Bit	1	0	1	**r8**	1	1	0	**r4**	1	**r2**	**r1**

2. Compute each redundancy bit with even parity:

Bit	Checks positions	Data bits there	Number of 1s	Parity (even)
r1	3, 5, 7, 9, 11	`1 0 1 1 1`	4	**0**
r2	3, 6, 7, 10, 11	`1 1 1 0 1`	4	**0**
r4	5, 6, 7	`0 1 1`	2	**0**
r8	9, 10, 11	`1 0 1`	2	**0**

3. Fill them in:

Position	11	10	9	8	7	6	5	4	3	2	1
Bit	1	0	1	**0**	1	1	0	**0**	1	**0**	**0**

Codeword = 10101100100

Q9. Received Hamming codeword 11001101101: what data was sent?

1. Write the received bits by position:

Position	11	10	9	8	7	6	5	4	3	2	1
Bit	1	1	0	**0**	1	1	0	**1**	1	**0**	**1**

2. Recompute each parity group, this time including its r bit:

Check	Positions (incl. r)	Received bits	Number of 1s	New parity
r1	1, 3, 5, 7, 9, 11	`1 1 0 1 0 1`	4	**0**
r2	2, 3, 6, 7, 10, 11	`0 1 1 1 1 1`	5	**1**
r4	4, 5, 6, 7	`1 0 1 1`	3	**1**
r8	8, 9, 10, 11	`0 0 1 1`	2	**0**

3. New parity bits r8 r4 r2 r1 = **0110** = **6**, so the bit in position 6 is wrong. Flip it from 1 to 0:

Position	11	10	9	8	7	6	5	4	3	2	1
Bit	1	1	0	**0**	1	**0**	0	**1**	1	**0**	**1**

4. Read the data from positions 11, 10, 9, 7, 6, 5, 3: `1 1 0 1 0 0 1`.

Data sent = 1101001 (single error at position 6, corrected)

Q10. Data 1110010: (a) single parity, (b) Hamming

(a) The data has four 1s, which is already even, so the parity bit is **0**. Codeword = `1110010 0`.

(b) Place the data:

Position	11	10	9	8	7	6	5	4	3	2	1
Bit	1	1	1	r8	0	0	1	r4	0	r2	r1

Bit	Checks positions	Data bits there	Number of 1s	Parity (even)
r1	3, 5, 7, 9, 11	`0 1 0 1 1`	3	**1**
r2	3, 6, 7, 10, 11	`0 0 0 1 1`	2	**0**
r4	5, 6, 7	`1 0 0`	1	**1**
r8	9, 10, 11	`1 1 1`	3	**1**

Position	11	10	9	8	7	6	5	4	3	2	1
Bit	1	1	1	**1**	0	0	1	**1**	0	**0**	**1**

(a) 11100100 · (b) 11110011001

Q11. Received 10111010110: detect and correct (at most one error, in the data bits)

Position	11	10	9	8	7	6	5	4	3	2	1
Bit	1	0	1	**1**	1	0	1	**0**	1	**1**	**0**

Check	Positions (incl. r)	Received bits	Number of 1s	New parity
r1	1, 3, 5, 7, 9, 11	`0 1 1 1 1 1`	5	**1**
r2	2, 3, 6, 7, 10, 11	`1 1 0 1 0 1`	4	**0**
r4	4, 5, 6, 7	`0 1 0 1`	2	**0**
r8	8, 9, 10, 11	`1 1 0 1`	3	**1**

1. New parity bits r8 r4 r2 r1 = **1001** = **9**. Position 9 is a data bit, which fits the question's assumption. Flip it from 1 to 0:

Position	11	10	9	8	7	6	5	4	3	2	1
Bit	1	0	**0**	**1**	1	0	1	**0**	1	**1**	**0**

2. Corrected codeword: `10011010110`. Data from positions 11, 10, 9, 7, 6, 5, 3: `1001011`.

Error at position 9 · corrected codeword 10011010110 · data 1001011